
Chapitre 4:

Couche liaison de données

Les codes correcteurs d'erreurs

Introduction

Nous avons étudié jusque là, tous les mécanismes à mettre en œuvre pour **transmettre** un **flot de bits** **entre deux systèmes distants**

Cependant, il ne suffit pas d'envoyer seulement l'information, mais il faut encore **contrôler l'échange de données**

On appelle protocole, un ensemble de conventions préétablies pour réaliser **un échange fiable de données** entre deux entités:

- ☐ le contrôle de l'**intégrité** des données reçues
- ☐ l'organisation et le **contrôle de l'échange**
- ☐ éventuellement le **contrôle de la liaison**



Introduction

▪ **Contrôle d'intégrité:**

Lors d'une transmission de données, on doit assurer que les données reçues n'ont pas été altérées durant la transmission (fiabilité de transmission)

Plusieurs facteurs peuvent modifier le contenu des données, les uns sont d'origine humaine:

→ le contrôle d'intégrité concerne alors la sécurité des données

Les autres sont d'origine physique:

→ le contrôle d'intégrité porte alors le nom de contrôle d'erreur

- Les rayonnements électromagnétiques
- Les perturbations propres au système (bruit...)



Peuvent modifier les Informations transmises (bits erronés)

Contrôle d'erreur

▪ Notion d'erreur

On appelle taux d'erreur binaire ou BER(Bit Error Rate) le rapport entre le nombre d'informations (bits) erronées reçues et le nombre d'informations (bits) transmises

$$\text{BER} = \text{Nb_bitsErronnés} / \text{Nb_bitsTransmises}$$

Exemple:

- La suite transmise est : « 011001001100100101001010 »
- La suite reçue est : « 011001101100101101000010 »
- Nombre de bites transmise est: 24
- Nombre de bites erroné est: 3

$$\text{Donc, BER} = 3/24 = 0,125$$

➔ Le taux d'erreur binaire varie en pratique de $10e-4$ à $10e-9$ (réseaux locaux)

Détection d'erreur

La détection d'erreur est les mécanismes mis en œuvre pour que le système destinataire puisse vérifier la validité des données reçues.
La détection d'erreur repose sur l'introduction d'une certaine redondance dans l'information transmise

Détection d'erreur

Méthodes mises en place au niveau de la couche 2 OSI (« **liaison de données** »)

Principe général :

- ❖ Chaque suite de bits (trame) à transmettre est augmentée par une autre **suite de bit dite de redondance ou de contrôle**.

- ❖ Pour chaque suite de k bits transmis, on ajoute r bits. On dit alors que l'on utilise **un code $C(n; k)$ avec $n = k + r$** .

À la réception, on effectue l'opération inverse et **les bits ajoutés permettent d'effectuer des contrôles à l'arrivée**.

→ **Quatre techniques** peuvent être mises en œuvre pour détecter et éventuellement corriger les erreurs:

- ❖ La détection par écho
- ❖ La détection par répétition
- ❖ La détection par clé calculée (VRC, CRC...)
- ❖ La détection et correction d'erreur par code (code de Hamming,...)

Détection d'erreur

▪Détection par écho:

Principe

Le récepteur renvoie en écho le message reçu à l'émetteur

Si le message est différent de celui émis, l'émetteur retransmet le message

Utilisation

Cette technique est utilisée dans les terminaux asynchrones (Telnet,...)

▪Détection par répétition :

Principe

Chaque message émis est suivi de sa réplique

Si les deux messages sont différents, le récepteur demande une retransmission

Utilisation

Cette technique est utilisée dans les milieux sécurisés très perturbés et dans certaines applications dites temps réel

Détection d'erreur

■ DÉTECTION PAR CLÉ CALCULÉE

Principe

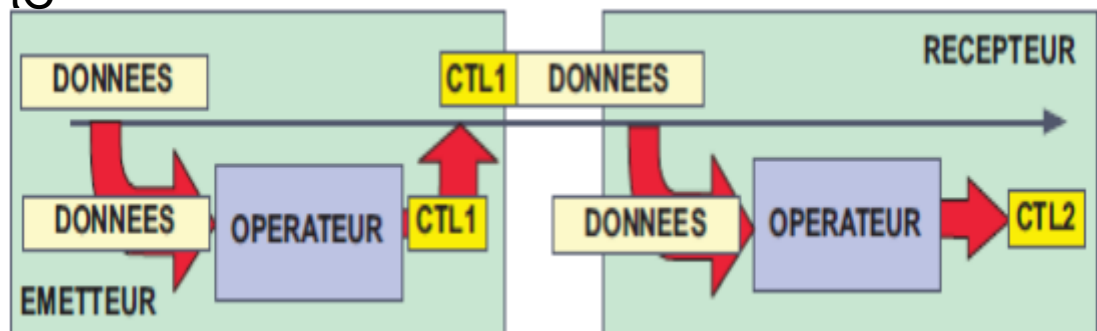
- Une information supplémentaire (clé) déduite des informations transmises est ajoutée à celles-ci
- L'émetteur calcule une séquence de contrôle **CTL1** par une opération mathématique appliquée au message à émettre et envoyée avec le message
- Le récepteur recalcule la clé **CTL2** en utilisant la même opération.
 - si CTL1 identique à CTL2, alors **le bloc est exact**:
 - sinon **le bloc est rejeté**

■ TECHNIQUES

Bit de parité ou checksum ou VRC

LRC

Codes cycliques



Détection d'erreurs par VRC

- **Vertical Redundancy Check** : ajout d'un bit de parité à chaque mot de code.
 - Il consiste à ajouter un bit supplémentaire (appelé **bit de parité**) à un certain nombre de bits de données appelé *mot de code* (généralement 7 bits, pour former un octet avec le bit de parité) dont la valeur (0 ou 1) est telle que le nombre total de bits à 1 soit pair.
 - Si le nombre de 1 dans le mot de code est impair, on ajoute un 1, sinon on ajoute un 0.
 - Vérification par redondance verticale ne permet de détecter que les erreurs portant sur un nombre impair de bits. Cette méthode détecte les **erreurs dans 50% des cas**.

Détection d'erreurs par LRC

Longitudinal Redundancy Check : à partir de blocs de caractères munis de leurs bits de parité, rangés dans les lignes d'un tableau, on calcule la parité des colonnes et on transmet le résultat avec le message.

- Consiste non pas à contrôler l'intégrité des données d'un caractère, mais à contrôler l'intégrité des bits de parité d'un bloc de caractères.
- Le LRC est lui-même protégé par un bit de parité VRC

Caractère à transmettre	bit de parité	Caractère à transmettre	bit de parité	...	Caractère LRC	bit de parité
-------------------------------	---------------------	-------------------------------	---------------------	-----	------------------	---------------------

Détection d'erreurs par LRC & VRC

- Soit « HELLO » le message à transmettre, en utilisant le code ASCII standard. Voici les données telles qu'elles seront transmises avec les codes de contrôle de parité croisé :
- Cette méthode détecte les erreurs **dans 90% des cas**.

	H	E	L	L	O	LRC →
bit 0	0	1	0	0	1	0
bit 1	0	0	0	0	1	1
bit 2	0	1	1	1	1	0
bit 3	1	0	1	1	1	0
bit 4	0	0	0	0	0	0
bit 5	0	0	0	0	0	0
bit 6	1	1	1	1	1	1
VRC ↓	0	1	1	1	1	0

1001000	0	1000101	1	1001100	1	1001100	1	1001111	1	1000010	0
H		E		L		L		O		LRC	

Détection d'erreurs par LRC

MSB \ LSB		0	1	2	3	4	5	6	7
		000	001	010	011	100	101	110	111
0	0000	NUL	DLE	SP	0	@	P	`	p
1	0001	SOH	DC1	!	1	A	Q	a	q
2	0010	STX	DC2	"	2	B	R	b	r
3	0011	ETX	DC3	#	3	C	S	c	s
4	0100	EOT	DC4	\$	4	D	T	d	t
5	0101	ENQ	NAK	%	5	E	U	e	u
6	0110	ACK	SYN	&	6	F	V	f	v
7	0111	BEL	ETB	'	7	G	W	g	w
8	1000	BS	CAN	(	8	H	X	h	x
9	1001	HT	EM	)	9	I	Y	i	y
A	1010	LF	SUB	*	:	J	Z	j	z
B	1011	VT	ESC	+	;	K	[	k	}
C	1100	FF	FS	,	<	L	\	l	
D	1101	CR	GS	-	=	M	]	m	{
E	1110	SO	RS	.	>	N	^	n	~
F	1111	SI	US	/	?	O	_	o	DEL

MSB
1=bit n°6 pour
coder caractère H

LSB
Premier bit de
droite=bit n°0 pour
coder caractère H

- Chaque caractère est codé par une association **MSB-LSB**

- Exp « N » = **100** **1110**

Code polynomiaux ou cyclic redundancy code (CRC)

- Redondance cyclique **CRC** (**Cyclic Redundancy Check**) parfois appelé aussi **FCS** (**Frame Check Sequence**) : Codes les plus utilisés (à la fois dans les LAN et WAN)

- **Idée de base**

- A toute séquence de « k » bits, on peut faire correspondre un polynôme de degré « $k - 1$ » et de coefficients « 0 » ou « 1 » (ou inversement)

Exemple: Soit la séquence de $k = 10$ bits **1 1 0 1 0 1 1 0 1 1**

Le Polynôme de degré $k - 1 = 9$ associé à cette séquence est donné par

$$1x^9 + 1x^8 + 0x^7 + 1x^6 + 0x^5 + 1x^4 + 1x^3 + 0x^2 + 1x + 1 = x^9 + x^8 + x^6 + x^4 + x^3 + x + 1$$

- Supposons que les données soient transmises par bloc de « k » bits

- *A chaque bloc correspond un polynôme $M(x)$ de degré « k »*

- *Exemple pratique*

$$1\ 1\ 0\ 1\ 0\ 1\ 1\ 0\ 1\ 1 \longrightarrow M(x) = x^9 + x^8 + x^6 + x^4 + x^3 + x + 1$$

- Au départ, un **émetteur** et un **récepteur** doivent choisir un polynôme générateur $G(x)$ de degré inférieur à « k »

- *Ce polynôme sera utilisé pendant toute la durée de la transmission*

- *Exemple de polynôme générateur $\rightarrow G(x) = x^4 + x + 1$*

- *Polynôme générateur normalisés par le C.C.I.T.T:*

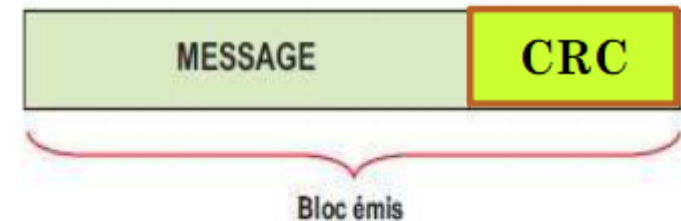
- » $G(x) = x^{16} + x^{12} + x^5 + 1$

- » *Permet de détecter des erreurs de transmission avec **une probabilité supérieure à 0.999***

Détection d'erreurs par le code CRC

▪ Codage à l'émission pour un bloc de « k » bits de données

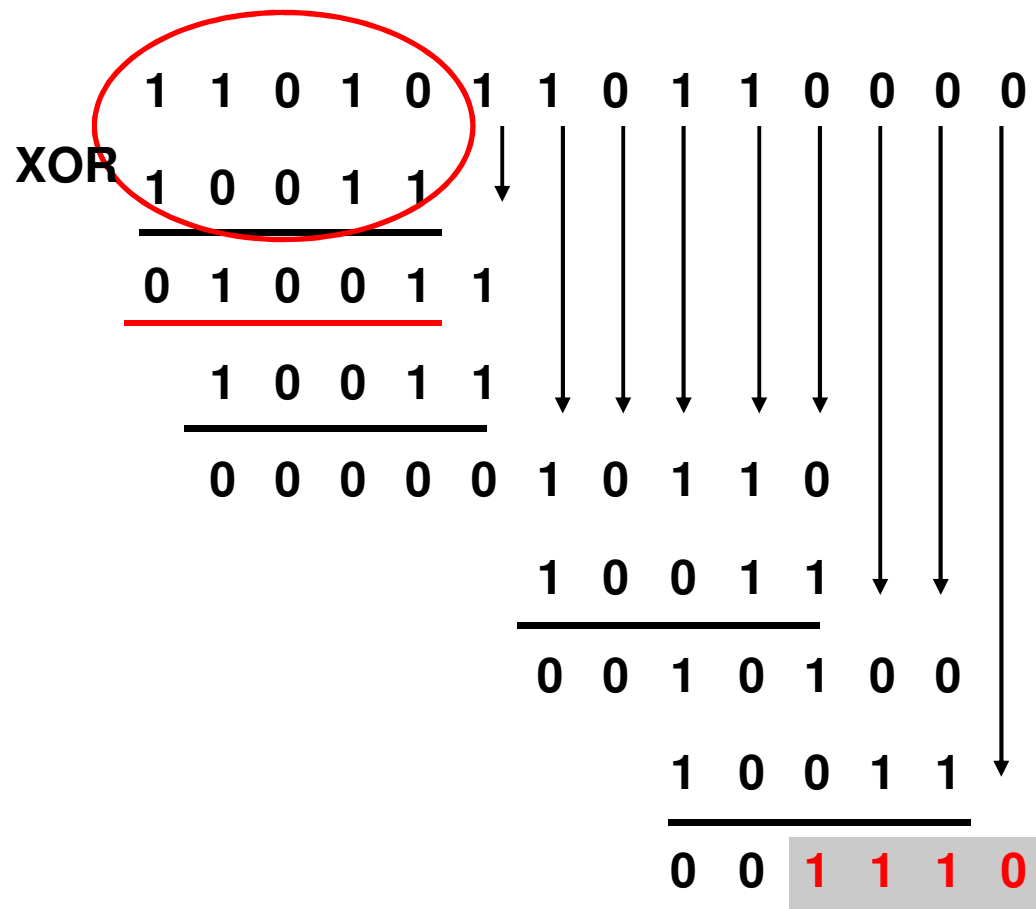
- Multiplier $M(x)$ par x^r où « r » est le degré de $G(x)$
- Calculer le reste $R(x)$ de la division modulo 2 de $x^r.M(x)$ par $G(x)$
- Ajouter $R(x)$ à $x^r.M(x)$
- Transmettre la séquence de « k + r » bits correspondante au polynôme $T(x) = x^r.M(x) + R(x)$



▪ Détection d'erreurs de transmission à la réception

- Soit $U(x)$ le polynôme correspondant à la séquence de « k + r » bits reçus
- Calculer le reste de la division modulo 2 de $U(x)$ par $G(x)$
- Si le reste est nul $\longrightarrow$ Pas d'erreur de transmission
- Si le reste est non nul $\longrightarrow$ Une erreur de transmission est détectée

▪ Exemple de codage CRC



1	0	0	1	1					
1	1	0	0	0	0	1	0	1	0

Opération XOR : OU exclusif

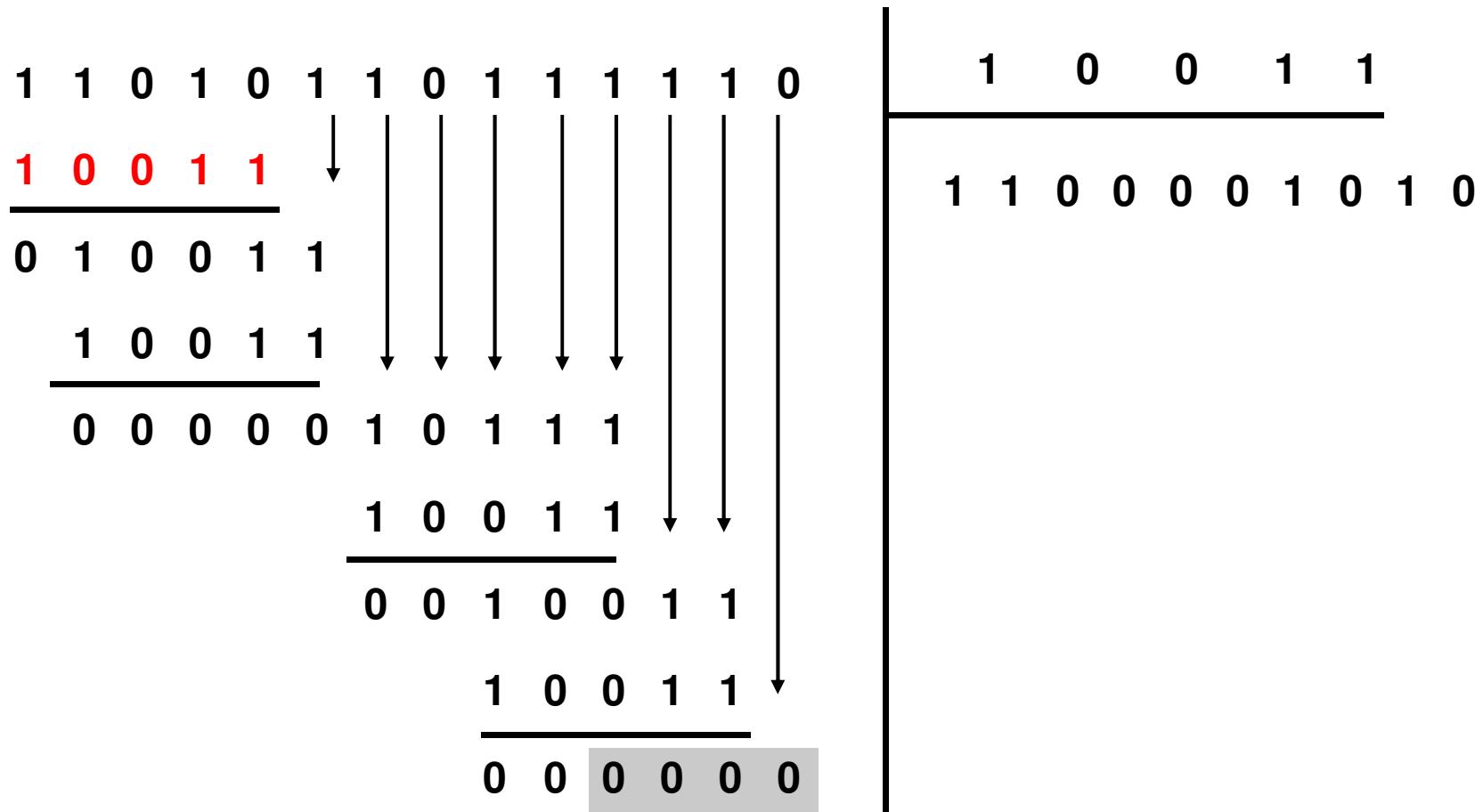
XOR	A B	S = A ⊕ B
	0 0	0
	0 1	1
	1 0	1
	1 1	0

1 1 0 1 0 1 1 0 1 1 1 1 1 0



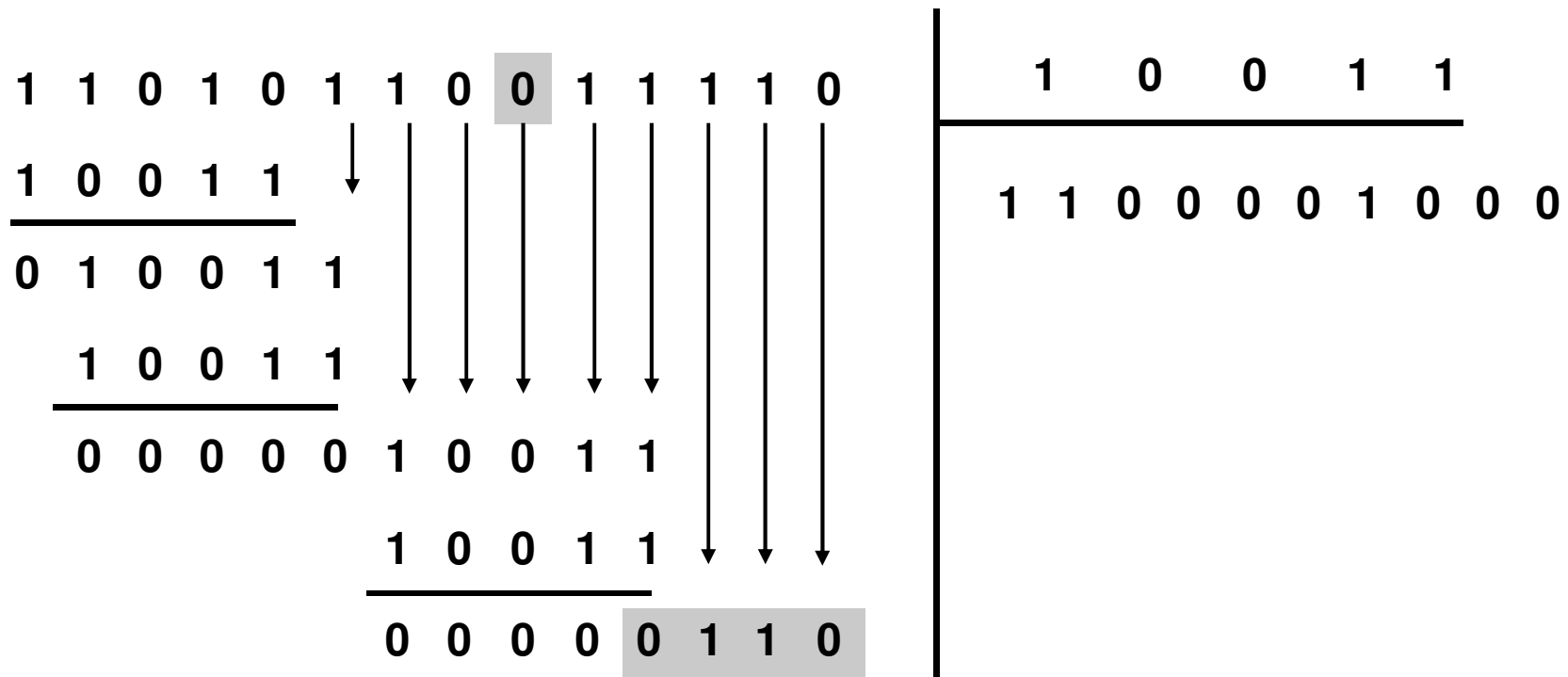
Bits transmis (mot code transmis)
Msg information + reste 16

▪ Exemple de décodage CRC (Sans erreur de transmission)



↑ Pas d'erreur car le reste est nul

Exemple de décodage CRC (Avec erreur de transmission)



Erreur de transmission
car le reste n'est pas nul

▪ Exemple:

Le plus simple est encore de prendre un exemple : prenons le message M de 14 bits suivant : *11010011101100*

Prenons $G(X) = X^3 + X + 1$ (représenté en binaire par *1011*).

Etant donné que $G(X)$ est de degré 3, il s'agit d'ajouter 3 bits nuls à M :

$M' = 11010011101100$ *000*.

Le CRC est égal au reste de la division de M' par G :

Je viens de recevoir le code CRC suivants 100, est-il correct ?

code de Hamming

Détection par code de Hamming

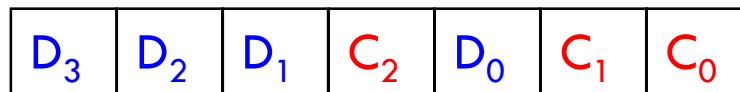
- **Structure d'un mot de code de Hamming:**
 - Le code de Hamming $C(n,k)$ est constitué des k bits d'information et des r bits de redondance (de contrôle): c'est la séquence qui va être transmise.
 - k appelé dimension du code = la taille du message d'information.
 - n appelé longueur du code = la taille du mot de code.

code de Hamming

- ❑ Les bits de contrôle C_i sont en position 2^i dans le mot de code transmis, .
- ❑ Les bits du message D_i occupe le reste du code.
- ❑ On appelle efficacité ou rendement ou taux d'un code le rapport:

$$R = \frac{k}{n}$$

- ❑ Exemple: Code C(7,4)
 - ❑ Efficacité=4/7=57%
 - ❑ Le mot de code est:



→ Sens de la transmission

code de Hamming

□ Construction du code de Hamming:

- On construit la matrice **H** appelée **matrice de contrôle** comme suit:

$$H = \begin{pmatrix} 0 & 0 & \dots & 1 \\ 0 & \vdots & \dots & \vdots \\ \vdots & 1 & \dots & \vdots \\ 1 & 0 & \dots & 1 \end{pmatrix} \left. \vphantom{\begin{pmatrix} 0 & 0 & \dots & 1 \\ 0 & \vdots & \dots & \vdots \\ \vdots & 1 & \dots & \vdots \\ 1 & 0 & \dots & 1 \end{pmatrix}} \right\} \begin{array}{l} (n-k) \text{ lignes} \\ n \text{ colonnes} \end{array}$$

- Le mot de code est tel que:

$$C \cdot H^t = 0$$

code de Hamming

□ Exemple: Code C(7,4)

$$H = \begin{pmatrix} 0001111 \\ 0110011 \\ 1010101 \end{pmatrix} \Rightarrow C \cdot H^t = (C_0 \ C_1 \ D_0 \ C_2 \ D_1 \ D_2 \ D_3) \cdot \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{pmatrix}$$

$$\Rightarrow \begin{cases} C_2 = D_1 + D_2 + D_3 \\ C_1 = D_0 + D_2 + D_3 \\ C_0 = D_0 + D_1 + D_3 \end{cases}$$

$$\Rightarrow \begin{cases} C_2 \text{ est calculé par rapport aux bits d'indices 5, 6, 7.} \\ C_1 \text{ est calculé par rapport aux bits d'indices 3, 6, 7.} \\ C_0 \text{ est calculé par rapport aux bits d'indices 3, 5, 7.} \end{cases}$$

code de Hamming

□ **Détection et correction d'erreurs avec le code de Hamming:**

- A la réception, on reçoit la séquence de code $\tilde{C}$. On effectue le produit:

$$\tilde{C} \cdot H^t$$

- Si le produit est nul, alors la séquence de code a été correctement transmise ($\tilde{C} = C$).
- Sinon, il y a des erreurs de transmission ($\tilde{C} \neq C$).
Et le résultat du produit indique **la position de l'erreur** en décimal.

code de Hamming

□ **Exercice 1:** On souhaite envoyer le message **1010**. Quel est le mot de de Hamming à transmettre ?

⇒ $k=4 \rightarrow n=7 \rightarrow$ On codera avec le code $C(7,4)$.

⇒ $C = D_3 D_2 D_1 C_2 D_0 C_1 C_0$
 $= 1 0 1 - 0 - -$

$$\text{On a : } \begin{cases} C_2 = D_1 + D_2 + D_3 = 1 + 0 + 1 = 0 \\ C_1 = D_0 + D_2 + D_3 = 0 + 0 + 1 = 1 \\ C_0 = D_0 + D_1 + D_3 = 0 + 1 + 1 = 0 \end{cases}$$

⇒ $C = 1 0 1 0 0 1 0$

code de Hamming

□ **Exercice 2:** Y-a-t-il une erreur dans le mot reçu suivant
1 0 1 0 1 1 0 ?

⇒ $n=7 \rightarrow$ le code qui a été utilisé est $C(7,4)$.

⇒ La matrice de parité est la suivante:

$$H = \begin{pmatrix} 0001111 \\ 0110011 \\ 1010101 \end{pmatrix}$$

⇒ En effectuant le produit $\tilde{C} \cdot H^t$, on trouve un reste non nul.

⇒ Il y a donc des erreurs.

code de Hamming

- ⇒ $\tilde{C} \cdot H^t = (001)$, c'est à dire 1 en base 10.
- ⇒ Il y a donc une erreur à la position 1 du mot reçu.
- ⇒ Le mot de code correct est: **1 0 1 0 1 1 1**

1^{er} bit corrigé

code de Hamming

▪ Application:

Soit un mot de Hamming de longueur 15

1	0	1	1	0	1	1	1	1	0	1	1	0	1	1
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1

- ☐ Quels sont les bits de contrôle de parité ?
- ☐ Quel est le message reçu ?
- ☐ Est-ce que le message reçu correspond au message transmis ?
- ☐ Quel a été le message transmis ?

code de Hamming

Application :

- Soit le code de Hamming reçu suivant de longueur 15:
101101111011011

- Quels sont les bits de contrôle de parité ?

1	0	1	1	0	1	1	1	1	0	1	1	0	1	1
D ₁₀	D ₉	D ₈	D ₇	D ₆	D ₅	D ₄	C ₃	D ₃	D ₂	D ₁	C ₂	D ₀	C ₁	C ₀

⇒ Les bits de contrôle sont: 1111.

- Quel est le message d'information reçu ?

⇒ Le message reçu est : 10110111010.

code de Hamming

□ Est-ce que le message reçu correspond au message transmis ?

⇒ La matrice de parité est de taille $H(n-k, n) = H(4, 15)$. Le produit $\tilde{C} \cdot H^t = (0000)$

⇒ Il n'y a pas d'une erreur dans le message reçu.